Raport tehnic: Aplicație Client–Server pentru Evaluarea Problemelor de Programare

Curmei Bianca

Facultatea de Informatică, Universitatea "Alexandru Ioan-Cuza" din Iași

Introducere

Acest raport descrie implementarea unei aplicații client–server în limbajul C, folosind socket-uri TCP și un model de concurență bazat pe procese (`fork()`). Aplicația permite clienților să solicite probleme de programare, să trimită soluții în C și să primească rezultate, calculând automat scorurile în funcție de corectitudinea soluției, timpul de execuție și viteza de trimitere a fișierelor.

Obiectivele proiectului sunt:
- înțelegerea comunicării prin socket-uri TCP;
- implementarea unui server concurent folosind `fork()` pentru mai mulți clienți simultan;
- crearea unui protocol simplu de nivel aplicație, bazat pe comenzi text;
- evaluarea automată a soluțiilor de programare în C.

Aplicația servește ca prototip pentru o platformă de evaluare automată a participanților la olimpiade de informatică.

1 Tehnologii Aplicate

1.1 Limbajul de programare

Proiectul este implementat în **C**, oferind acces direct la socket-uri, control asupra memoriei și execuției, și integrare ușoară cu scripturi bash pentru compilare și testare.

1.2 Protocolul de transport

Comunicarea dintre client și server se realizează prin **TCP**, care asigură:
- comunicare fiabilă;
- garantarea ordinii mesajelor;
- menținerea unei conexiuni persistente potrivită pentru dialog interactiv.

Serverul ascultă pe portul 5151.

1.3 API-ul de socket-uri

Se utilizează API-ul standard Berkeley Sockets:
- `socket()`, `bind()`, `listen()`, `accept()`;
- `connect()`, `send()`, `recv()`.

1.4 Modelul de concurență

Serverul folosește concurență prin procese:
- procesul părinte ascultă permanent conexiuni noi;
- la acceptarea unui client, se creează un proces copil cu `fork()`;
- procesul copil gestionează comunicarea cu clientul: trimiterea problemelor și evaluarea soluțiilor.

2 Structura Aplicației

Aplicația este împărțită în două programe principale: `server` și `client`, împreună cu scripturi auxiliare pentru compilare și rularea testelor. Structura aplicației și responsabilitățile fiecărui component sunt detaliate mai jos.

2.1 Structura directoarelor generale

```
problems/
  problem1/
    statement.txt
    tests/
      test1.in, test1.out, ...
submissions/
results/
scripts/
```

2.2 Structura Serverului

Fișiere:
- `server.c` - gestionează conexiunile clienților și logica principală;
- `problem_manager.c/h` - încarcă problemele și testele din directoarele locale;
- `evaluator.c/h` - compilează și rulează soluțiile, calculează scorurile;
- `utils.c/h` - funcții de ajutor pentru trimiterea/primirea mesajelor și prelucrarea inputului;
- scripturile `compile_solution.sh` și `run_tests.sh` pentru compilare și testare automată.

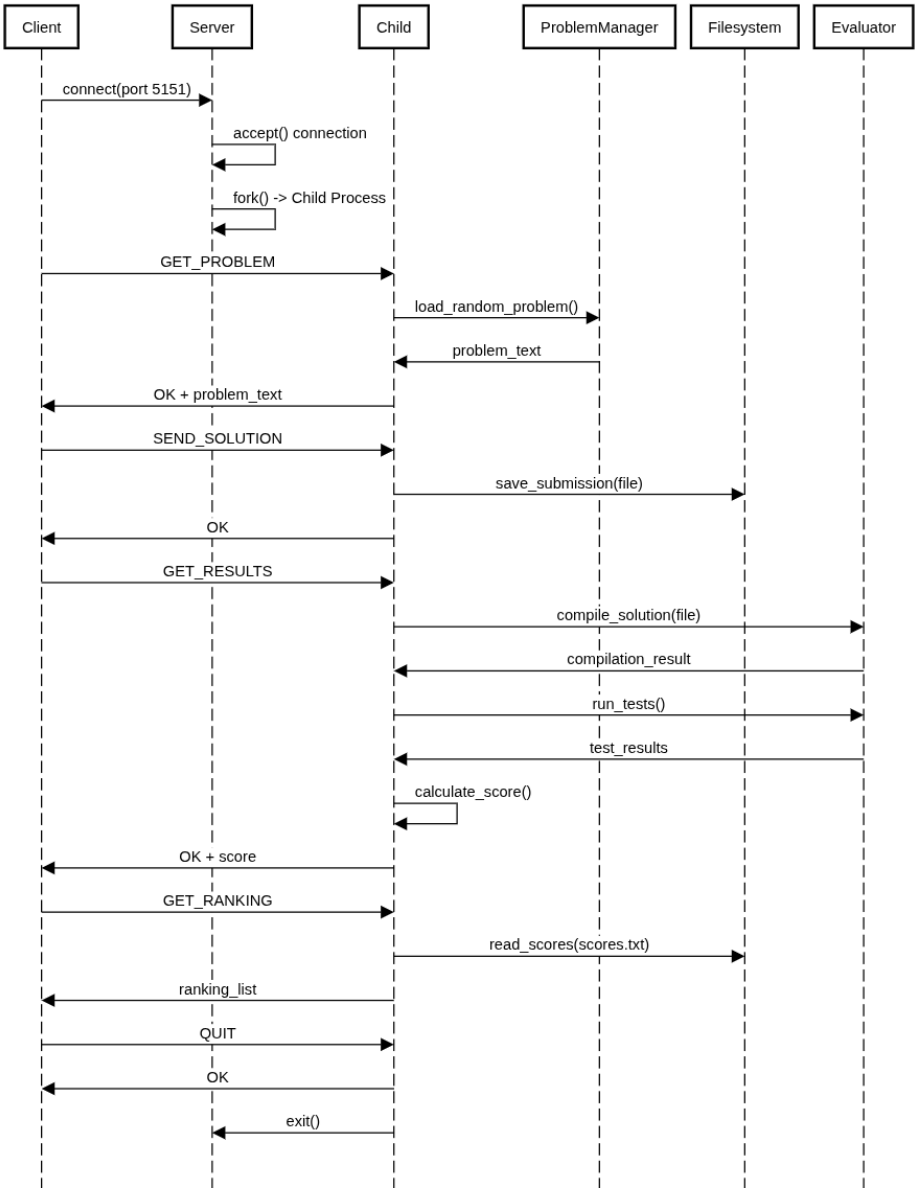
Funcționalitate:
- primește conexiuni TCP de la clienți pe portul 5151;
- la fiecare client, creează un proces copil (`fork()`) care gestionează întreaga sesiune;
- implementează comenzile:
  - `GET PROBLEM` - trimite enunțul unei probleme aleatoare;
  - `SEND SOLUTION` - primește soluția clientului și o salvează în `submissions/`;
  - `GET RESULTS` - calculează și trimite scorul final;
  - `GET RANKING` - trimite lista tuturor participanților și scorurile lor;
  - `QUIT` - închide conexiunea cu clientul.

2.3 Structura Clientului

Fișiere:
- `client.c` - programul principal pentru interacțiunea utilizatorului;
- `utils.c/h` - funcții utilitare pentru comunicare și prelucrarea inputului.

Funcționalitate: – se conectează la serverul local pe portul 5151;
– citește comenzile de la utilizator și le trimite serverului;
– afișează răspunsurile serverului: probleme, scoruri, clasament.

2.4 Diagrama de comunicare



Explicație detaliată a diagramei secvențiale:

- **Conectarea clientului la server:** Clientul inițiază o conexiune TCP către server pe portul 5151. Serverul principal acceptă conexiunea și creează un proces copil folosind `fork()` pentru a gestiona această sesiune.

- **Solicitarea unei probleme (`GET_PROBLEM`):** Clientul trimite comanda `GET_PROBLEM` către procesul copil. Copilul solicită un enunț aleatoriu din modulul `ProblemManager`, care îl încarcă din directorul `problems/`. Problema este trimisă clientului împreună cu confirmarea `OK`.

- **Trimiterea soluției (`SEND_SOLUTION`):** Clientul trimite fișierul sursă al soluției. Procesul copil salvează soluția în directorul `submissions/` și confirmă primirea cu `OK`.

- **Evaluarea rezultatelor (`GET_RESULTS`):** Clientul solicită scorul. Copilul compilează soluția folosind modulul `Evaluator`, rulează testele stocate în directorul problemei, colectează rezultatele și calculează scorul final. Scorul este trimis clientului împreună cu `OK`.

- **Obținerea clasamentului (`GET_RANKING`):** Clientul solicită clasamentul. Procesul copil citește fișierul `results/scores.txt`, care conține scorurile tuturor participanților, și trimite lista clientului.

- **Închiderea conexiunii (`QUIT`):** Clientul închide sesiunea trimițând comanda `QUIT`. Procesul copil confirmă cu `OK` și se termină, eliberând resursele alocate.

- **Separarea proceselor și modulelor:** Serverul principal rămâne disponibil pentru alte conexiuni, în timp ce fiecare client activ este gestionat independent de un proces copil. Acest model asigură concurență și izolare între sesiunile clienților.

3 Protocolul de Comunicare

3.1 Comenzi client

Comandă	**Descriere**
GET PROBLEM	Solicită o problemă aleatorie.
SEND SOLUTION	Trimite o soluție în C.
GET RESULTS	Solicită rezultatul evaluării.
GET RANKING	Solicită clasamentul tuturor participanților.
QUIT	Închide conexiunea.

3.2 Răspunsuri server

- **OK** - comandă procesată cu succes;
- **ERR** - comandă invalidă sau eroare.

3.3 Exemplu de dialog

```
Client: GET_PROBLEM
Server: OK
Server: Problema 3: Sortare prin interclasare.
Client: SEND_SOLUTION
Server: OK
Client: GET_RESULTS
Server: OK 95
Client: QUIT
Server: OK
```

4 Scenarii de Utilizare

4.1 Scenarii de succes

Scenariul 1: Client cere o problemă și apoi închide conexiunea.

1. Clientul se conectează la server.
2. Trimite `GET PROBLEM`; serverul răspunde cu `OK` + problemă.
3. Trimite `QUIT`; serverul confirmă cu `OK`.
4. Clientul închide conexiunea.

Scenariul 2: Flux complet.

1. Clientul se conectează.
2. Trimite `GET PROBLEM`; serverul trimite problema.
3. Trimite `SEND SOLUTION`; serverul confirmă.
4. Trimite `GET RESULTS`; serverul trimite scorul calculat.
5. Trimite `GET RANKING`; serverul trimite clasamentul.
6. Trimite `QUIT`; serverul confirmă.

Scenariul 3: Mai mulți clienți simultan.

1. Mai mulți clienți se conectează.
2. `accept()` creează socket dedicat fiecărui client.
3. `fork()` creează un proces copil per client.
4. Fiecare proces copil gestionează independent propriul client.

4.2 Scenarii de eșec

Scenariul 1: Comandă invalidă.
1. Clientul trimite `HELLO`.
2. Serverul răspunde cu `ERR`.
3. Clientul decide să încerce din nou sau să închidă conexiunea.

Scenariul 2: Serverul se închide brusc.
1. Clientul este conectat.
2. Serverul se oprește neașteptat.
3. Clientul trimite o comandă și apare eroare de citire/scriere.
4. Clientul închide aplicația.

Scenariul 3: Portul 5151 este ocupat.
1. `bind()` eșuează.
2. Serverul afișează eroare și se închide.
3. Clientul vede `connection refused`.

5 Concluzii

Aplicația demonstrează modul de funcționare al unui sistem client–server bazat pe TCP și concurență prin procese. Codul este simplu, robust și ușor de extins.

Posibile îmbunătățiri:
- comunicare criptată TLS pentru securitate;
- trecerea la model concurent pe thread-uri sau epoll pentru scalabilitate mare;
- definirea unui protocol structurat (JSON/XML) pentru extensibilitate si interoperabilitate;
- interfata grafica sau web pentru utilizatori

Referințe

1. Documentația Linux: `man socket`, `man bind`, `man listen`.
2. Documentația `fork(2)`.
3. Springer LNCS Author Guidelines: `https://www.springer.com/gp/computer-science/lncs/conference-proceedings-guidelines`.
4. Pagina cursului Rețele de Calculatoare: `https://edu.info.uaic.ro/computer-networks/cursullaboratorul.php`.